BLOQUE III - TEMA 1

DESARROLLO DE SISTEMAS

Modelado de datos, metodologías y reglas. Entidades, atributos y relaciones. Diseño de bases de datos. Diseño lógico y físico. El modelo lógico relacional. Normalización.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. MODELO CONCEPTUAL DE DATOS 7

1.1. Herramientas para la construcción del Modelo Conceptual de Datos 7

1.2. Importancia en el ciclo de vida de los sistemas 8

1.3. Relación con el modelo lógico y físico 9

1.4. Arquitectura ANSI/SPARC de Tres Niveles 9

2. ENTIDADES, ATRIBUTOS Y RELACIONES. ELEMENTOS DEL MODELO CONCEPTUAL 10

2.1. Entidades 10

2.2. Atributos: clasificación y notación 11

2.3. Relaciones 13

3. MODELADO DE DATOS 15

4. REGLAS Y METODOLOGÍAS EN EL MODELADO DE DATOS. REGLAS DE CONSTRUCCIÓN 16

4.1. Reglas de Integridad 16

4.2. Restricciones y Claves. Reglas de dependencia 18

5. DISEÑO DE BASES DE DATOS 20

5.1. Fases del Diseño de Bases de Datos 20

5.2. Diseño Conceptual, Lógico y Físico 21

5.2.1. Diseño Conceptual 21

5.2.2. Diseño Lógico 22

5.2.3. Diseño Físico 23

6. OPERACIONES DEL ÁLGEBRA RELACIONAL 25

6.1 Operaciones fundamentales (primitivas) 25

6.2. Operaciones derivadas 26

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7. EL MODELO LÓGICO RELACIONAL 26

7.1. Definición y Características 26

7.2. Tablas, Claves Primarias y Foráneas 27

7.2.1. Tablas en el Modelo Relacional 27

7.2.2. Clave Primaria (Primary Key - PK) 28

7.2.3. Clave Foránea (Foreign Key - FK) 29

7.3. Relaciones y Dependencias 29

7.3.1. Tipos de Relaciones 29

7.3.2. Dependencias en el Modelo Relacional 30

8. MODELO ENTIDAD/RELACIÓN EXTENDIDO (EER) Y SU APLICACIÓN EN BASES DE DATOS 31

8.1. Definición y características 31

8.1.1. Herencia 31

8.1.2. Entidades (entidades fuertes y débiles) 31

8.1.3. Agregación 32

8.1.4. Relaciones más complejas: superclases y subclases 32

8.1.5. Atributos 32

8.1.6. Relaciones y cardinalidad 32

8.2. Herencia. Generalización y Especialización en el modelo E-R extendido 33

8.2.1. Generalización 34

8.2.2. Especialización 34

8.2.3. Entidades asociativas y de agregación en el Modelo E/R extendido 35

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

9. NORMALIZACIÓN 37

9.1. Normalización de Bases de Datos. Concepto y objetivos 38

9.2. Formas Normales 38

9.2.1. Primera Forma Normal (1FN) 39

9.2.2. Segunda Forma Normal (2FN) 39

9.2.3. Tercera Forma Normal (3FN) 40

9.2.4. Forma Normal de Boyce-Codd (BCNF) 41

9.2.5. Cuarta Forma Normal (4FN) 42

9.2.6. Quinta Forma Normal (5FN) o Forma Normal de Proyección-Unión 42

9.3. Dependencia funcional 43

9.4. Dependencia funcional transitiva 44

9.5. Dependencia de reunión (Join Dependency) 45

9.6. Ejemplo análisis Formas Normales 46

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. MODELO CONCEPTUAL DE DATOS

El **modelo conceptual de datos** es una representación abstracta, clara e independiente de la tecnología que describe **los datos necesarios para el funcionamiento de un sistema**, junto con sus relaciones y restricciones. Forma parte de la **fase de análisis** en el desarrollo de sistemas y constituye la **base para el diseño lógico y físico** de bases de datos.

Se centra en **qué datos existen**, **cómo se relacionan entre sí** y **qué reglas deben cumplirse**, sin preocuparse aún de cómo se almacenarán en un sistema de gestión de bases de datos (SGBD).

El **modelo conceptual de datos** es una representación **abstracta, lógica y estructurada** de los elementos que componen un sistema de información. Su objetivo es **describir los datos relevantes del sistema, las relaciones entre ellos y las reglas de integridad**, sin hacer referencia a aspectos físicos o de implementación.

Elementos principales del Modelo Conceptual

Elemento	Descripción
Entidad	Representa un objeto, concepto o evento del mundo real del que se desea guardar información. Ej.: Cliente, Producto, Curso.
Atributo	Propiedad o característica de una entidad. Ej.: nombre, fecha_nacimiento, precio.
Clave primaria	Atributo o conjunto de atributos que identifica de forma única una instancia de la entidad.
Relación	Asociación entre dos o más entidades que tiene significado en el dominio del sistema. Ej.: Cliente realiza Pedido.
Cardinalidad	Indica cuántas veces una entidad puede participar en una relación (1:1, 1:N, N:M).
Entidad débil	Entidad que no tiene clave primaria propia y depende de otra para su existencia.
Atributos derivados	Aquellos que se pueden calcular a partir de otros. Ej.: edad a partir de fecha_nacimiento.

1.1. HERRAMIENTAS PARA LA CONSTRUCCIÓN DEL MODELO CONCEPTUAL DE DATOS

Para crear un **modelo conceptual de datos** preciso, estructurado y comprensible, es fundamental apoyarse en herramientas de modelado que permitan representar visual y formalmente las entidades, relaciones y restricciones del sistema.

a. Modelo Entidad-Relación (E-R)

- Propuesto por Peter Chen en 1976, es el **modelo más utilizado para el diseño conceptual de bases de datos relacionales**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Elementos básicos**:
  - » **Entidades** representadas como rectángulos.
  - » **Relaciones** como rombos conectados a las entidades.
  - » **Atributos** como óvalos unidos a las entidades o relaciones.
  - » **Claves primarias** subrayadas.
- **Aplicaciones**: diseño de bases de datos, docencia, documentación técnica.

b. UML (Unified Modeling Language)

- Lenguaje de modelado estándar propuesto por OMG (Object Management Group), ampliamente utilizado en ingeniería del software orientada a objetos.
- **Utilización en bases de datos**: Se usa principalmente el diagrama de clases para representar entidades, atributos, relaciones y cardinalidades.
- **Elementos básicos**:
  - » Clases: equivalen a entidades, con atributos y operaciones.
  - » Asociaciones: representan relaciones entre clases.
  - » Multiplicidad: especifica la cardinalidad de las relaciones.

1.2. IMPORTANCIA EN EL CICLO DE VIDA DE LOS SISTEMAS

El modelo conceptual de datos tiene un papel **crítico** dentro del **ciclo de vida del desarrollo de sistemas**, especialmente en la **fase de análisis** y como puente hacia el diseño.

- Garantiza que los datos del sistema están **completos, normalizados y libres de redundancia**.
- Facilita la **comunicación entre analistas, diseñadores y desarrolladores**.
- Es clave para construir:
  - » El **modelo lógico** (adaptado al tipo de base de datos).
  - » El **modelo físico** (estructuras concretas de almacenamiento).
- Mejora la **calidad, mantenibilidad y escalabilidad** del sistema.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.3. RELACIÓN CON EL MODELO LÓGICO Y FÍSICO

El desarrollo de bases de datos y sistemas de información se realiza de manera **progresiva** a través de diferentes niveles de abstracción:

Modelo	Nivel de abstracción	Características
Modelo conceptual	Alto	Independiente del SGBD. Describe entidades, relaciones y reglas del negocio.
Modelo lógico	Medio	Adapta el modelo conceptual a un modelo de base de datos específico (relacional, orientado a objetos, etc.).
Modelo físico	Bajo	Describe cómo se almacenan físicamente los datos: índices, particiones, estructuras.

1.4. ARQUITECTURA ANSI/SPARC DE TRES NIVELES

La **arquitectura ANSI/X3/SPARC** fue propuesta por el comité ANSI en los años 70 como un modelo de referencia para la organización y diseño de sistemas de bases de datos. Su objetivo principal es **separar los distintos niveles de abstracción**, facilitando la independencia de los datos, la seguridad y la gestión eficiente.

Esta arquitectura jerárquica se divide en **tres niveles fundamentales**:

1. Nivel Externo (Esquemas Externos)

- Representa las **vistas personalizadas de la base de datos** que tienen los diferentes usuarios o aplicaciones.
- Cada vista muestra únicamente **una parte de la base de datos**, adaptada a las necesidades específicas de quien la usa.
- Permite ocultar información irrelevante o sensible y **controlar el acceso**.
- Es posible definir múltiples vistas externas sobre un mismo esquema conceptual.

Ejemplo: Un administrativo del departamento de recursos humanos solo ve los nombres y puestos de los empleados, pero no sus salarios ni datos personales completos.

2. Nivel Conceptual (Esquema Conceptual)

- Es el **nivel intermedio** que describe la estructura lógica general de la base de datos para toda la organización.
- Incluye las **entidades, relaciones, atributos y restricciones de integridad**.
- Define **qué datos existen** y **cómo se relacionan**, sin preocuparse aún por cómo se almacenan físicamente.
- Aquí se ubica el **modelo lógico de datos**, que actúa como puente entre el modelo conceptual y el físico. Puede estar basado en modelos relacionales, orientados a objetos, jerárquicos, etc.

Ejemplo: El esquema conceptual define tablas como EMPLEADOS, DEPARTAMENTOS y sus relaciones, como qué campos se conectan y cuáles son claves primarias o foráneas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

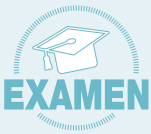
3. Nivel Interno o Físico (Esquema Interno)

- Detalla **cómo se almacenan los datos físicamente en el sistema**: estructuras de archivos, índices, compresión, partición, etc.
- Se encarga de **optimizar el rendimiento y uso del almacenamiento**.
- Este nivel es invisible para los usuarios finales y contiene detalles técnicos dependientes del SGBD.

Ejemplo: Cómo se guardan realmente las filas de la tabla EMPLEADOS en disco, qué índices se crean para acelerar consultas, o si los datos están cifrados.

2. ENTIDADES, ATRIBUTOS Y RELACIONES. ELEMENTOS DEL MODELO CONCEPTUAL

El modelo conceptual de datos se construye a partir de componentes fundamentales que permiten describir de forma clara los objetos del sistema y cómo se relacionan.



Estos elementos son: **entidades, atributos, relaciones, restricciones**, entre otras.

2.1. ENTIDADES

Una **entidad** es cualquier objeto del mundo real o abstracto que puede ser representado en una base de datos y sobre el cual se almacena información. Puede ser un objeto físico, un concepto o un evento.

Tipos de entidades:

- ***Fuertes o independientes***: poseen una **clave primaria propia**. Ej.: Empleado, Producto.
- ***Débiles o dependientes***: **no tienen clave primaria propia** y dependen de otra entidad. Se identifican por su relación con una entidad fuerte. Ej.: LíneaFactura, DetallePedido.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Representación en diagramas E-R:

Las entidades se representan en diagramas mediante un rectángulo con su nombre en su interior.

- **Entidades fuertes**: rectángulo de línea simple con el nombre dentro.
- **Entidades débiles**: rectángulo de línea doble.



Entidad fuerte



Entidad débil

Ejemplos de Entidades

- En un sistema de gestión de estudiantes, algunas entidades podrían ser:
  - **Estudiante** (representa a una persona inscrita en una institución).
  - **Curso** (representa una materia ofrecida).
  - **Profesor** (persona encargada de impartir un curso).
- En un sistema de comercio electrónico:
  - **Cliente** (usuario que realiza compras).
  - **Pedido** (orden de compra realizada por un cliente).
  - **Producto** (artículo disponible para la venta).

2.2. ATRIBUTOS: CLASIFICACIÓN Y NOTACIÓN

Los **atributos** son las **características** o propiedades que describen a una entidad. Cada entidad tiene un conjunto de atributos que definen sus propiedades únicas.

1. **Atributos Simples o Atómicos**:
   - Son aquellos que **no** pueden dividirse en partes más pequeñas.
   - Ejemplo: *Nombre*, *Edad*, *Precio*.
2. **Atributos Compuestos**:
   - Son aquellos que **pueden dividirse** en subatributos más pequeños.
   - Ejemplo: *Dirección* puede dividirse en *Calle, Número, Ciudad, Código Postal*.

© Reservados todos los derechos de esta edición para OPOVERDE EDUCACIÓN S.L., 2025

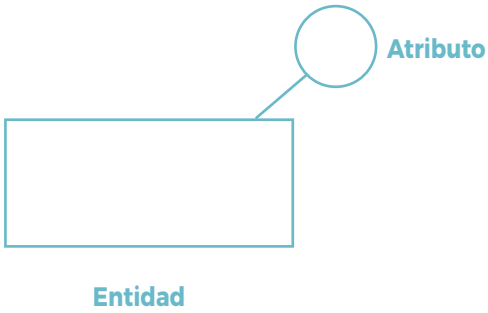
3. **Atributos Clave o Identificadores**:
    - Son aquellos que permiten **identificar de manera única** a cada entidad en una base de datos.
    - Ejemplo: *ID de estudiante*, *Número de pasaporte*.
4. **Atributos Derivados**:
    - Son aquellos que **pueden obtenerse a partir de otros atributos.**
    - Ejemplo: *Edad* puede derivarse de la *Fecha de Nacimiento*.
5. **Atributos Multivaluados**:
    - Son aquellos que **pueden tener más de un valor para una entidad.**
    - Ejemplo: Un *profesor* puede tener múltiples *títulos académicos*.

Notación en diagramas E-R:

- Atributo: **óvalo pequeño** unido a la entidad o relación.
- Atributo clave: **óvalo relleno**.
- Atributo compuesto: **óvalo** conectado a subatributos.
- Atributo multivaluado: **óvalo** de **línea doble**.

EXAMEN

En los diagramas Entidad-Relación (E-R), los atributos se representan con óvalos conectados a la entidad correspondiente.



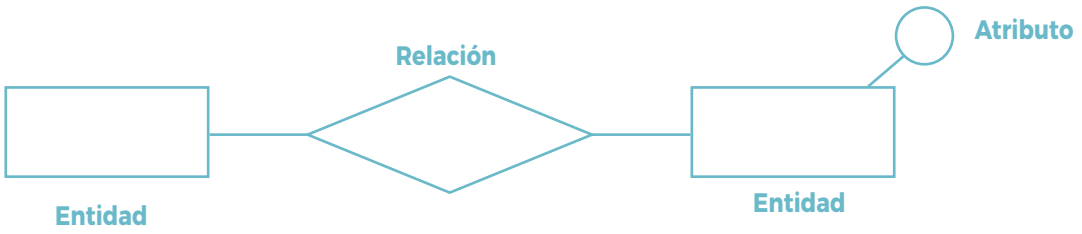
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.3. RELACIONES

Las **relaciones representan asociaciones entre entidades**. Permiten establecer conexiones y reglas de negocio dentro de una base de datos.

1. **Relación Uno a Uno (1:1)**
    - Una entidad A se asocia con una única entidad B y viceversa.
    - Ejemplo: Un **pasaporte** pertenece a un solo **ciudadano**, y cada **ciudadano** tiene un solo **pasaporte**.
2. **Relación Uno a Muchos (1:M)**
    - Una entidad A puede estar relacionada con varias entidades B, pero una entidad B solo puede estar relacionada con una entidad A.
    - Ejemplo: Un **profesor** puede impartir varios **cursos**, pero cada **curso** tiene un solo **profesor**.
3. **Relación Muchos a Muchos (M:N)**
    - Una entidad A puede estar relacionada con varias entidades B y viceversa.
    - Ejemplo: Un **estudiante** puede estar inscrito en varios **cursos**, y un **curso** puede tener varios **estudiantes** inscritos.
    - Para modelar este tipo de relación en bases de datos relacionales, se utiliza una **tabla intermedia**.

Las relaciones se representan en los diagramas E-R con un rombo, donde las entidades conectadas se enlazan mediante líneas.



Grado de una relación: El grado de una relación indica el número de entidades que participan en una misma relación. Es un concepto clave en el diseño del modelo conceptual, ya que define la estructura de las asociaciones entre los objetos del sistema.

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Grado	Descripción	Ejemplo
Unitario	Una sola entidad participa en la relación consigo misma (**relación recursiva**).	Un Empleado supervisa a otro.
Binario	Dos entidades diferentes están involucradas (el caso más común).	Un Cliente realiza un Pedido.
Ternario	Involucra tres **entidades diferentes** en una única relación.	Un Profesor evalúa a un Alumno en un Curso.

Cardinalidad: Indica **cuántas instancias** de una entidad pueden estar asociadas a una instancia de otra entidad en la relación.

1. **Cardinalidad Mínima**: Indica si la participación de la entidad en la relación es obligatoria o no. Puede ser **0** (opcional) o **1** (obligatoria).
2. **Cardinalidad Máxima**: Define el número máximo de instancias que pueden participar en la relación.

Ejemplo para una relación "Profesor - Curso":

- (1,1) → Un profesor debe impartir **exactamente** un curso.
- (1,M) → Un profesor debe impartir **al menos** un curso, pero puede impartir varios.
- (0,M) → Un profesor **puede o no** impartir cursos, pero si lo hace, puede impartir varios.

***Dependencia*:** Las **dependencias** establecen restricciones en las relaciones entre entidades:

1. **Dependencia Débil**
   - Ocurre cuando una entidad no puede existir sin otra.
   - Ejemplo: Un **pedido** no puede existir sin un **cliente**.
   - Se representa con un doble rectángulo en el modelo E-R.
2. **Dependencia Fuerte**
   - Se da cuando una entidad puede existir sin depender de otra.
   - Ejemplo: Un **cliente** puede existir en el sistema sin haber realizado pedidos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. MODELADO DE DATOS

El **modelado de datos** es una **disciplina** esencial en la gestión y diseño de bases de datos. Consiste en **representar de manera estructurada la información de un sistema**, facilitando su organización, almacenamiento y recuperación eficiente.

El **modelado de datos es el proceso de estructurar y organizar la información en un sistema de bases de datos** mediante la identificación de **entidades**, **atributos** y **relaciones**. Se representa a través de modelos conceptuales, lógicos y físicos para garantizar la coherencia y eficiencia en el manejo de los datos.

Existen diversas metodologías utilizadas en el modelado de datos, cada una con enfoques específicos según el tipo de sistema y los requerimientos. A continuación, se hace un repaso de las metodologías existentes.

1. Modelo Entidad-Relación (E-R)

- Representa los datos mediante **entidades (objetos del sistema), atributos (características de las entidades) y relaciones (conexiones entre entidades)**.
- Se visualiza con diagramas E-R.

2. Modelo Relacional

- Organiza los datos en **tablas con filas y columnas**, donde cada fila representa una entidad y cada columna un atributo.
- Define relaciones mediante **claves primarias y foráneas**, estableciendo conexiones entre tablas.

3. Modelado Orientado a Objetos

- Extiende el enfoque de bases de datos tradicionales, incorporando **clases, objetos y herencia**.
- Es utilizado en sistemas donde se requiere almacenar estructuras más complejas.

4. Modelado Dimensional

- Se emplea en bases de datos para **almacenamiento y análisis de datos históricos**(Data Warehouse).
- Utiliza estructuras como **esquema estrella o copo de nieve**, optimizando consultas para análisis de grandes volúmenes de datos.

5. Modelado NoSQL

- Aplicado en bases de datos NoSQL como MongoDB o Cassandra, donde los datos se organizan en **documentos, grafos o pares clave-valor**.
- Ideal para **aplicaciones escalables y de alto rendimiento**, como redes sociales o big data.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4. REGLAS Y METODOLOGÍAS EN EL MODELADO DE DATOS. REGLAS DE CONSTRUCCIÓN

El modelado de datos no solo implica definir entidades, atributos y relaciones, sino que también requiere la aplicación de reglas que aseguren la **coherencia, integridad y eficiencia** en la estructura de una base de datos.

4.1. REGLAS DE INTEGRIDAD

Las reglas de integridad garantizan que los datos almacenados en la base de datos sean **correctos, consistentes y válidos.**

1. Integridad de Entidad



La **integridad de entidad** garantiza que **cada fila (tupla) en una tabla pueda ser identificada de forma única** mediante una **clave primaria (PK, Primary Key)**.

Reglas:

- Toda entidad debe tener **una clave primaria definida**.
- La clave primaria **no puede contener valores nulos.**
- Los **valores en la clave primaria deben ser únicos**: no puede haber dos registros con la misma clave. No pueden ser duplicados -> **Identificador único**

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo:

En la tabla Clientes, el campo id_cliente actúa como clave primaria. Ningún cliente puede tener el mismo id_cliente, ni puede dejarse en blanco.

```
CREATE TABLE Clientes (
  id_cliente INT PRIMARY KEY,
  nombre VARCHAR(50),
  email VARCHAR(100)
);
```

2. Integridad Referencial

La **integridad referencial** se encarga de **mantener la coherencia entre entidades relacionadas**, a través del uso de **claves foráneas (FK, Foreign Keys)**.

Reglas:

- Garantiza que las **relaciones entre entidades sean válida**s y que **no existan datos huérfanos.**
- Se implementa mediante **claves foráneas**, que deben hacer referencia a una clave primaria existente en otra tabla.
- Una entidad relacionada con otra mediante una **clave foránea** debe referenciar una instancia existente.

Ejemplo:

Una tabla **Pedidos** contiene el campo **id_cliente,** que **debe hacer referencia a un id_cliente válido en la tabla Clientes.**

```
CREATE TABLE Pedidos (
  id_pedido INT PRIMARY KEY,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. Integridad de Dominio

La **integridad de dominio** asegura que los valores introducidos en un campo sean **válidos en relación con el tipo de datos y restricciones definidas para ese atributo**.

Reglas:

- Los valores almacenados en los atributos deben pertenecer a un conjunto válido de valores predefinidos.
- Los valores de cada atributo deben pertenecer a un **dominio válido predefinido** (tipo de dato, formato, rango).
- Se aplica mediante **tipos de datos, validaciones y restricciones**.
- Ejemplo: Un campo "Edad" en una tabla de empleados no puede contener valores negativos.

Ejemplo:

Un campo edad definido como entero positivo no debe aceptar valores negativos ni texto.

```
CREATE TABLE Personas (
  id INT PRIMARY KEY,
  nombre VARCHAR(50),
  edad INT CHECK (edad >= 0 AND edad <= 120)
);
```

4. Integridad de Negocio

- Se basa en reglas específicas definidas por la lógica del negocio o aplicación.

Tipo de integridad	¿Qué garantiza?
Entidad	Identificación única de cada registro.
Referencial	Coherencia entre tablas relacionadas.
Dominio	Que los valores almacenados sean correctos y esperados.

4.2. RESTRICCIONES Y CLAVES. REGLAS DE DEPENDENCIA

Las **restricciones** son condiciones impuestas sobre los datos para garantizar su validez. Algunas de las más utilizadas en bases de datos relacionales incluyen:

1. **NOT NULL**: Impide que un campo tenga valores nulos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. **UNIQUE**: Garantiza que un valor en una columna no se repita.
3. **CHECK**: Permite definir reglas personalizadas para validar los datos.
4. **DEFAULT**: Especifica un valor predeterminado para una columna si no se proporciona un valor explícito.

Tipos de Claves



Las **claves** son fundamentales en la estructura de las bases de datos, ya que permiten identificar registros y establecer relaciones entre tablas.

1. **Clave Primaria (Primary Key - PK)**
   - Es un atributo o conjunto de atributos que identifican de forma única cada registro en una tabla.
   - No puede contener valores nulos ni repetidos.
   - Ejemplo: "ID_Empleado" en la tabla "Empleados".
2. **Clave Foránea (Foreign Key - FK)**
   - Es un atributo que establece una relación con otra tabla, haciendo referencia a su clave primaria.
   - Garantiza la integridad referencial.
   - Ejemplo: "ID_Cliente" en la tabla "Pedidos" hace referencia a "ID_Cliente" en la tabla "Clientes".
3. **Clave Candidata**
   - Son atributos que pueden servir como clave primaria, pero solo se elige una.
   - Ejemplo: En una tabla de estudiantes, "Matrícula" y "DNI" podrían ser claves candidatas, pero solo una se elige como clave primaria.
4. **Clave Alternativa**
   - Es una clave candidata que no ha sido seleccionada como clave primaria.
   - Ejemplo: Si "Matrícula" es la clave primaria en la tabla "Estudiantes", el "DNI" sería una clave alternativa.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. **Clave Compuesta**
    - Se forma combinando dos o más atributos para crear un identificador único.
    - Ejemplo: En una tabla de "Inscripciones", la clave primaria podría ser la combinación de "ID_Estudiante" y "ID_Curso".

5. DISEÑO DE BASES DE DATOS

El diseño de bases de datos es un proceso fundamental para la construcción de sistemas de información eficientes, organizados y escalables. Consiste en definir la estructura que tendrá la base de datos, asegurando la correcta organización y almacenamiento de los datos.

El diseño de una base de datos se lleva a cabo en **varias fases**, desde la identificación de los requisitos hasta la implementación del modelo físico.

5.1. FASES DEL DISEÑO DE BASES DE DATOS

El proceso de diseño de bases de datos se divide en varias etapas, cada una con un propósito específico para garantizar que la estructura final sea eficiente y optimizada.

Fase 1: Análisis de Requisitos

- Se identifican las necesidades del sistema y los datos que se van a gestionar.
- Se entrevista a los usuarios y expertos para comprender qué información se debe almacenar y cómo se va a utilizar.
- Se definen las restricciones y reglas de negocio.

Fase 2: Diseño Conceptual

- Se crea una representación abstracta de los datos, generalmente mediante el **Modelo Entidad-Relación (E-R)**.
- Se identifican las **entidades, atributos y relaciones**.
- Se determina la **cardinalidad** y se establecen las restricciones de integridad.

Fase 3: Diseño Lógico

- Se transforma el modelo conceptual en un **modelo relacional** o en el esquema correspondiente al tipo de base de datos elegido.
- Se definen las **tablas, claves primarias y foráneas, atributos y relaciones**.
- Se aplica el proceso de **normalización** para reducir la redundancia y mejorar la eficiencia del almacenamiento.

Fase 4: Diseño Físico

- Se implementa el modelo lógico en un **sistema de gestión de bases de datos (DBMS)** específico.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Se optimiza el almacenamiento mediante la elección de **tipos de datos, índices y particionamiento**.
- Se establecen **políticas de seguridad y permisos de acceso**.

Fase 5: Implementación y Pruebas

- Se crean las estructuras de la base de datos en el DBMS.
- Se insertan datos de prueba para verificar que el diseño es funcional y eficiente.

5.2. DISEÑO CONCEPTUAL, LÓGICO Y FÍSICO

El **modelo ANSI/SPARC** es una arquitectura propuesta por el Instituto Nacional Estadounidense de Estándares (ANSI) en colaboración con el Comité de Planificación de Sistemas de Automatización de Procesos Reales (SPARC), que define una estructura de tres niveles para la gestión de bases de datos.

Su objetivo principal es **separar la lógica de los datos de su almacenamiento físico** y permitir distintas vistas adaptadas a cada tipo de usuario. Esta separación proporciona una mayor independencia entre los programas de aplicación y la organización de los datos, facilitando la administración, la seguridad, el mantenimiento y la evolución del sistema.



Los tres niveles definidos —**externo**, **conceptual** e **interno**— representan respectivamente la visión del usuario, la estructura lógica global del sistema y la forma en que los datos se almacenan físicamente.

5.2.1. Diseño Conceptual

El **diseño conceptual** es la primera fase en el proceso de diseño de bases de datos. Su objetivo principal es **representar la información que manejará el sistema de forma abstracta**, sin entrar en detalles técnicos o de implementación.

Objetivos:

- Capturar y describir **qué datos se van a manejar**, **cómo se relacionan** y **qué reglas deben cumplirse**, desde un punto de vista **independiente del sistema gestor de bases de datos (DBMS)**.
- Servir como **puente entre los requisitos del usuario y el diseño técnico posterior**.

Elementos clave:

Durante esta fase se construye el **modelo Entidad-Relación (E-R)**, que permite representar gráficamente:

- **Entidades**: objetos o conceptos del mundo real relevantes para el sistema (por ejemplo, Cliente, Producto, Pedido).
- **Atributos**: propiedades que describen a las entidades (nombre, precio, fecha).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Relaciones**: asociaciones entre entidades (por ejemplo, un Cliente realiza un Pedido).
- **Restricciones**: reglas sobre cardinalidades, participación, claves primarias y foráneas.

Qué no se define en esta fase:

- **No se especifica cómo se almacenan los datos** (ni tablas, índices, tipos de datos concretos).
- **No se elige un DBMS** ni se determina la estructura física del almacenamiento.
- Tampoco se entra en la optimización del rendimiento ni en estructuras internas.

5.2.2. Diseño Lógico



El **diseño lógico** es la fase del desarrollo de bases de datos en la que se **transforma el modelo conceptual** en una estructura **compatible con un sistema de gestión de bases de datos relacional (DBMS)**, aunque **todavía sin definir aspectos físicos ni específicos del motor**.

Objetivos:

- **Traducir el modelo conceptual a un esquema lógico** que se pueda implementar fácilmente en un SGBD relacional (como MySQL, PostgreSQL, SQL Server, etc.).
- Garantizar que la base de datos resultante **mantenga la integridad, eficiencia y ausencia de redundancia**, utilizando para ello herramientas como la **normalización**.

Principales transformaciones:

Durante esta fase, se realizan las siguientes acciones:

- **Entidades** → se convierten en **tablas**.
- **Atributos** → se convierten en **columnas** de esas tablas.
- **Relaciones** → se convierten en **tablas adicionales o claves foráneas**, dependiendo de la cardinalidad.
- **Claves primarias** → se definen explícitamente para identificar de forma única cada fila.
- **Claves foráneas** → se establecen para representar las relaciones entre entidades y garantizar la **integridad referencial**.
- **Dominios** → se transforman en tipos de datos lógicos (por ejemplo, VARCHAR, INT, DATE...).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Normalización:

Durante el diseño lógico, se aplican las **reglas de normalización** (1FN, 2FN, 3FN, BCNF...) con el objetivo de:

- Eliminar **redundancias** de datos innecesarias.
- Evitar **anomalías de inserción, actualización o eliminación**.
- Asegurar que **los datos estén organizados de forma coherente**.

5.2.3. Diseño Físico

El **diseño físico** es la fase final en el proceso de diseño de bases de datos. En ella se **implementa el diseño lógico** en un **sistema de gestión de bases de datos específico (SGBD)**, teniendo en cuenta detalles técnicos concretos como el **almacenamiento**, el **rendimiento** y la **seguridad**.

Transformar el esquema lógico en una estructura real y funcional dentro de un SGBD (por ejemplo, **MySQL**, **Oracle**, **PostgreSQL**, **SQL Server**...), teniendo en cuenta:

- Las **características del sistema** donde se ejecutará.
- Los **requisitos de rendimiento, disponibilidad y mantenimiento**.
- Las **necesidades de acceso, control y seguridad** de los usuarios.

Actividades principales del diseño físico:

Durante esta fase, se toman decisiones específicas sobre cómo se implementará cada elemento de la base de datos:

» **Tipado de datos:**

- Se eligen los **tipos de datos físicos concretos** para cada columna según el SGBD - Sistema Gestor de Base de Datos (ej. VARCHAR, INT, DATE, DECIMAL, etc.).

Categoría	Tipo de dato	Descripción
Texto	CHA	Cadena de longitud fija (n caracteres).
	VARCHAR	Cadena de longitud variable (hasta n caracteres).
	TEXT / CLOB	Texto largo (dependiendo del SGBD).
Numéricos	INT / INTEGER	Entero (puede tener variantes: SMALLINT, BIGINT).
	DECIMAL	Número decimal con precisión (p) y escala (s).
	NUMERIC	Equivalente a DECIMAL, usado por compatibilidad.
	FLOAT / REAL	Números de punto flotante (menos precisión).
	DOUBLE PRECISION	Número de punto flotante de doble precisión.
Fechas y horas	DATE	Fecha (año, mes, día).
	TIME	Hora (hora, minutos, segundos).
	TIMESTAMP	Fecha y hora combinadas.
	INTERVAL	Intervalo de tiempo (solo en algunos SGBD como PostgreSQL).
Booleanos	BOOLEAN	Verdadero o falso (TRUE, FALSE, NULL).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Se definen **longitudes, precisión y restricciones adicionales** (NOT NULL, DEFAULT, UNIQUE, etc.).

Restricción	Descripción
NOT NULL	La columna no puede contener valores nulos.
DEFAULT	Asigna un valor por defecto si no se especifica uno al insertar un registro.
UNIQUE	Asegura que todos los valores en la columna sean únicos.
PRIMARY KEY	Define la columna como clave primaria (única y no nula).
FOREIGN KEY	Establece una relación con una columna de otra tabla (clave foránea).
CHECK	Establece una condición lógica que deben cumplir los valores (ej. CHECK (edad > 0)).
AUTO_INCREMENT/SERIAL	Genera automáticamente valores incrementales (según el SGBD).
INDEX	Crea un índice para acelerar búsquedas (no es una restricción en sí, pero se aplica sobre columnas).

» **Índices**

- Se crean **índices** sobre claves primarias y columnas frecuentemente consultadas para **mejorar el rendimiento** de búsqueda y ordenación.
- Pueden ser **índices simples o compuestos**, y en algunos casos se definen como **únicos**.

» **Almacenamiento**

- Se planifica **la distribución de tablas, índices y datos** en discos o particiones.
- Se definen aspectos como **compresión, particionamiento, segmentación** y uso de **espacios de tablas**.

» **Seguridad y permisos**

- Se configuran **usuarios, roles y privilegios de acceso** a los objetos de la base de datos.
- Se establecen políticas de **auditoría, autenticación y cifrado** si es necesario.

» **Optimización**

- Se realizan ajustes basados en la **carga esperada del sistema** (consultas frecuentes, concurrencia, crecimiento de datos...).
- Se pueden definir **vistas materializadas, triggers, procedimientos almacenados**, etc.

Técnicas y prácticas para optimizar el modelo físico



Cuando se quiere **reducir o simplificar el número de accesos** a la base de datos, especialmente en entornos con muchas consultas y operaciones frecuentes, se pueden aplicar las siguientes técnicas:

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

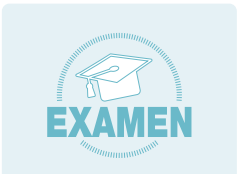
Técnica	Descripción
Introducir redundancia controlada	Se permite duplicar ciertos datos (que ya están normalizados) para evitar múltiples accesos o JOINs costosos. Mejora el rendimiento a cambio de espacio.
Combinar entidades	En lugar de mantener entidades separadas (como dicta la normalización), se pueden unir en una sola tabla si son accedidas juntas con frecuencia.
Desnormalización	Inverso de la normalización: se rompen algunas reglas para mejorar la eficiencia de lectura.
Uso de índices	Crear índices sobre campos clave o de búsqueda frecuente para acelerar accesos.
Materialización de vistas	Guardar resultados precomputados de consultas complejas para acceso más rápido.
CHECK	Establece una condición lógica que deben cumplir los valores (ej. CHECK (edad > 0)).
AUTO_INCREMENT/SERIAL	Genera automáticamente valores incrementales (según el SGBD).
INDEX	Crea un índice para acelerar búsquedas (no es una restricción en sí, pero se aplica sobre columnas).

6. OPERACIONES DEL ÁLGEBRA RELACIONAL

El **álgebra relacional** es un lenguaje formal que permite **definir consultas** sobre una base de datos relacional, a través de un conjunto de **operaciones sobre relaciones (tablas)**. Estas operaciones devuelven siempre una nueva relación como resultado (es **cerrado**).

6.1 OPERACIONES FUNDAMENTALES (PRIMITIVAS)

Las operaciones mínimas necesarias para construir cualquier otra operación más compleja en álgebra relacional son las de selección, proyección y unión, entre otras.



Operación	Símbolo	Descripción
Selección	σ	Filtra filas que cumplen una condición (ej. σ salario > 30000 (Empleado))
Proyección	π	Devuelve solo ciertas columnas (atributos) de la tabla
Unión	U	Une dos relaciones con el mismo esquema, eliminando duplicados
Diferencia	–	Devuelve las tuplas que están en una relación y no en otra
Producto cartesiano	×	Combina cada tupla de una relación con cada tupla de otra
Renombramiento	ρ	Permite cambiar el nombre de una relación o de sus atributos

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6.2. OPERACIONES DERIVADAS

Estas no son fundamentales, ya que se pueden construir a partir de las fundamentales:

Operación derivada	Descripción
Intersección	Se obtiene mediante la diferencia: A ∩ B = A − (A − B)
Join (Unión natural)	Combinación específica basada en igualdad de atributos
División	Se usa para consultas tipo "para todos", y se construye con varias operaciones básicas



El álgebra relacional es un lenguaje formal para consultar y manipular bases de datos relacionales. Sus operaciones fundamentales incluyen selección, proyección, unión, diferencia, producto cartesiano y renombramiento. Es la base teórica de SQL y permite construir consultas complejas de forma precisa y estructurada (que se estudiará en el siguiente tema)

7. EL MODELO LÓGICO RELACIONAL

El **modelo lógico relacional** es la base de las bases de datos relacionales y proporciona un enfoque estructurado para organizar y gestionar los datos. En este modelo, la información se representa mediante **tablas**, y las relaciones entre los datos se establecen a través de **claves primarias y foráneas**.

Este modelo es ampliamente utilizado en los **Sistemas de Gestión de Bases de Datos Relacionales (RDBMS)**, como MySQL, PostgreSQL, SQL Server y Oracle.

7.1. DEFINICIÓN Y CARACTERÍSTICAS

El **modelo relacional** es un modelo lógico que representa los datos en forma de **tablas** (también llamadas **relaciones**), donde cada fila corresponde a una **tupla** y cada columna a un **atributo**.

Características:

1. **Estructura basada en tablas**
    - La información se organiza en tablas, cada una con un nombre único.
    - Cada tabla representa una entidad y sus atributos.
2. **Uso de Claves Primarias y Foráneas**
    - La **clave primaria (Primary Key - PK)** identifica de forma única cada fila en una tabla.
    - La **clave foránea (Foreign Key - FK)** establece relaciones entre tablas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. **Independencia lógica y física**
    - El modelo lógico está separado del almacenamiento físico, lo que permite cambios en la estructura sin afectar la aplicación.
4. **Integridad y consistencia de datos**
    - Se imponen reglas de **integridad referencial** para garantizar que las relaciones entre tablas sean válidas.
5. **Manipulación de datos mediante SQL**
    - Las bases de datos relacionales utilizan **SQL (Structured Query Language)** para realizar consultas, inserciones, actualizaciones y eliminaciones de datos.
6. **Reducción de redundancia**
    - Se aplican técnicas como la **normalización** para evitar la duplicación de datos y mejorar la eficiencia.

Resumen elementos Modelo Lógico Relacional

Elemento	Descripción
Relación (Tabla)	Conjunto de tuplas que comparten la misma estructura de atributos.
Atributo	Columna de la relación; representa una propiedad o característica del dato.
Tupla	Fila de la tabla; representa un registro único.
Dominio	Conjunto de valores válidos que puede tomar un atributo (tipo + restricciones).
Clave primaria	Atributo(s) que identifican unívocamente una tupla en una relación.
Clave foránea	Atributo que referencia la clave primaria de otra relación.

7.2. TABLAS, CLAVES PRIMARIAS Y FORÁNEAS

7.2.1. Tablas en el Modelo Relacional

Las **tablas** son la estructura fundamental en el modelo relacional. Contienen datos organizados en:

- **Filas (tuplas)**
- **Columnas (atributos)**.

ID_Estudiante	Nombre	Apellido	Fecha_Nacimiento	Correo
1	Juan	Pérez	2000-05-12	juan.perez@email.com
2	María	López	1999-08-25	maria.lopez@email.com
3	Carlos	Sánchez	2001-02-18	carlos.sanchez@email.com

En este ejemplo, en la tabla Estudiantes, en cada fila se representa un estudiante único, y cada columna representa una propiedad de los estudiantes.

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Reglas del modelo lógico relacional:

1. Cada relación tiene un **nombre único**.
2. No puede haber **tuplas duplicadas** en una relación.
3. Los atributos deben tener **nombres únicos dentro de una relación**.
4. El orden de las tuplas no importa.
5. El valor de un atributo puede ser **atómico o NULL** (aunque NULL se usa con cuidado).

7.2.2. Clave Primaria (Primary Key - PK)



La **clave primaria** es un atributo o conjunto de atributos que identifican de manera única a cada registro en una tabla.

Características de la Clave Primaria:

- No puede contener valores **nulos**.
- No puede repetirse (debe ser **única** en la tabla).
- Solo puede haber **una clave primaria** por tabla.

Ejemplo anterior sobre la tabla **Estudiantes**:

- La columna **ID_Estudiante** es la clave primaria porque identifica de forma única a cada estudiante.



Una **clave primaria** debe cumplir dos condiciones fundamentales:

- **Unicidad**: no puede haber dos filas con el mismo valor en la clave.
- **No nulidad**: los valores de la clave primaria no pueden ser nulos (NOT NULL).

Cada tabla puede tener una única clave primaria, elegida entre sus claves candidatas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7.2.3. Clave Foránea (Foreign Key - FK)

Una **clave foránea** es un atributo que establece una relación con una clave primaria en otra tabla.

Características de la Clave Foránea:



- Garantiza la **integridad referencial**.
- Puede aceptar valores repetidos si la relación es **uno a muchos (1:M)**.
- Puede contener valores nulos si la relación no es obligatoria.

Ejemplo con dos tablas relacionadas, la tabla Estudiantes, y la nueva tabla Cursos:

- En la tabla **Estudiantes**, la columna **ID_Curso** es una **clave foránea** que hace referencia a la clave primaria **ID_Curso** en la tabla **Cursos**.
- Esto indica que cada estudiante está inscrito en un curso específico.

Tabla Estudiantes

ID_Estudiante	Nombre	Apellido	ID_Curso
1	Juan	Pérez	101
2	María	López	102
3	Carlos	Sánchez	101

Tabla Cursos

ID_Curso	Nombre_Curso
101	Matemáticas
102	Historia

7.3. RELACIONES Y DEPENDENCIAS

Las bases de datos relacionales permiten establecer **relaciones** entre diferentes tablas mediante **claves primarias y foráneas**.

7.3.1. Tipos de Relaciones

1. **Relación Uno a Uno (1:1)**
    - Una entidad A se asocia con una única entidad B y viceversa.
    - Ejemplo: Un **pasaporte** pertenece a un solo **ciudadano**, y cada **ciudadano** tiene un solo **pasaporte**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. **Relación Uno a Muchos (1:M)**
    - Una entidad A puede estar relacionada con varias entidades B, pero una entidad B solo puede estar relacionada con una entidad A.
    - Ejemplo: Un **profesor** puede impartir varios **cursos**, pero cada **curso** tiene un solo **profesor**.
3. **Relación Muchos a Muchos (M:N)**
    - Una entidad A puede estar relacionada con varias entidades B y viceversa.
    - Ejemplo: Un **estudiante** puede inscribirse en varios **cursos**, y un **curso** puede tener varios **estudiantes** inscritos.
    - Para modelar esta relación en bases de datos relacionales, se crea una **tabla intermedia**.

7.3.2. Dependencias en el Modelo Relacional

Las dependencias determinan cómo se relacionan los atributos dentro de una tabla.

1. **Dependencia Funcional**
    - Un atributo depende funcionalmente de otro si su valor está determinado por el valor de otro atributo.
    - Ejemplo: En la tabla **Estudiantes**, el atributo **Nombre** depende funcionalmente de **ID_Estudiante**.
2. **Dependencia Transitiva**
    - Se produce cuando un atributo A depende de otro atributo B, y este a su vez depende de otro atributo C.
    - Ejemplo: Si **Ciudad** depende de **Código_Postal**, y **Código_Postal** depende de **Dirección**, entonces hay una dependencia transitiva.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8. MODELO ENTIDAD/ RELACIÓN EXTENDIDO (EER) Y SU APLICACIÓN EN BASES DE DATOS

El **Modelo Entidad/Relación Extendido (EER)** es una extensión del **Modelo Entidad/Relación (E-R)** tradicional, diseñado para representar mejor los datos en bases de datos complejas. Este modelo introduce nuevos conceptos que permiten modelar información con mayor precisión y expresividad, especialmente cuando se requiere manejar **jerarquías, herencia, agregación y relaciones más avanzadas.**

8.1. DEFINICIÓN Y CARACTERÍSTICAS

En el contexto del **modelado de bases de datos**, la Métrica v3 proporciona un marco metodológico que ayuda a estructurar correctamente la información y definir relaciones entre entidades. Se basa en el **Modelo Entidad/Relación Extendido (EER)** para garantizar la correcta organización y normalización de los datos, facilitando la implementación en bases de datos relacionales.

8.1.1. Herencia

En el modelo EER, las entidades especializadas **heredan los atributos y relaciones** de sus entidades padre. Esto significa que no es necesario duplicar definiciones comunes, lo que **fomenta la reutilización y la claridad**.

> *Ejemplo: Si Persona tiene los atributos nombre y DNI, tanto Profesor como Estudiante los heredan automáticamente.*

8.1.2. Entidades (entidades fuertes y débiles)



El modelo distingue entre:

- **Entidades fuertes**: aquellas que tienen existencia propia y clave primaria.
- **Entidades débiles**: dependen de otra entidad para su existencia y no tienen clave propia. Se identifican mediante una clave parcial y una relación de identificación con su entidad fuerte.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Factura (entidad débil) puede depender de Cliente (entidad fuerte), ya que no tiene sentido una factura sin cliente.

8.1.3. Agregación

La **agregación** permite tratar una **relación como si fuera una entidad**, útil cuando una relación necesita participar en otras relaciones. Este mecanismo amplía la capacidad de modelado para estructuras más complejas.

Ejemplo: Si existe la relación TrabajaEn entre Empleado y Proyecto, se puede agregar como entidad para asociarla a otra como Evaluación.

8.1.4. Relaciones más complejas: superclases y subclases

El modelo EER soporta relaciones complejas como:

- **Superclases**: entidades generales que agrupan atributos comunes.
- **Subclases**: entidades especializadas con atributos o relaciones adicionales.

Esto permite representar **estructuras jerárquicas similares a las de la programación orientada a objetos**, facilitando la comprensión y mantenimiento del modelo.

8.1.5. Atributos

Una **entidad** representa un **objeto del mundo real** que tiene existencia independiente y características propias. Estas características se representan mediante **atributos**, que pueden ser:

- **Simples** o **compuestos**.
- **Monovalorados** o **multivalorados**.
- **Derivados** (calculables a partir de otros).

Ejemplo: La entidad Empleado puede tener los atributos nombre, fechaNacimiento, teléfonos (multivalorado), y edad (derivado de la fecha de nacimiento).

8.1.6. Relaciones y cardinalidad

Las **relaciones** vinculan entidades entre sí, cada relación puede tener una **cardinalidad** que indica el número de ocurrencias que una entidad puede tener asociadas en la relación.

- **1:1 (Uno a uno)**: Cada ocurrencia de una entidad está relacionada con **una sola ocurrencia** de otra entidad, y viceversa. Ejemplo: *Un empleado tiene un único jefe*.
- **1:N (Uno a muchos)**: Una entidad A puede estar relacionada con **muchas ocurrencias** de la entidad B, pero cada ocurrencia de B **solo se asocia con una** de A. Ejemplo: *Un departamento puede tener muchos empleados, pero cada empleado pertenece a un solo departamento.*
- **N:M (Muchos a muchos)**: Varias ocurrencias de una entidad pueden estar asociadas con varias ocurrencias de otra. Ejemplo: *Un empleado puede trabajar en varios proyectos y un proyecto puede tener varios empleados.*

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



En el **modelo Entidad/Relación Extendido (EER)** que utiliza **MÉTRICA v3**, una **relación** entre entidades se define formalmente por los siguientes elementos:

1. **Nombre de la relación**: identifica de forma única la relación dentro del modelo.
2. **Cardinalidad**: define el número mínimo y máximo de ocurrencias de una entidad que pueden estar asociadas a otra en la relación. Por ejemplo: (1,1), (0,N), etc.
3. **Tipo de correspondencia**: indica la naturaleza de la relación según el número de ocurrencias: uno a uno (1:1), uno a muchos (1:N) o muchos a muchos (N:M).

8.2. HERENCIA. GENERALIZACIÓN Y ESPECIALIZACIÓN EN EL MODELO E-R EXTENDIDO

La **herencia** en bases de datos permite que las subclases (por ejemplo, **Gerente**, **Técnico**) hereden atributos y relaciones de la superclase (por ejemplo, **Empleado**). Esto se representa con claves foráneas en la base de datos relacional.

Generalización y **especialización** son los procesos mediante los cuales se **detecta** o **se define** la relación de herencia.

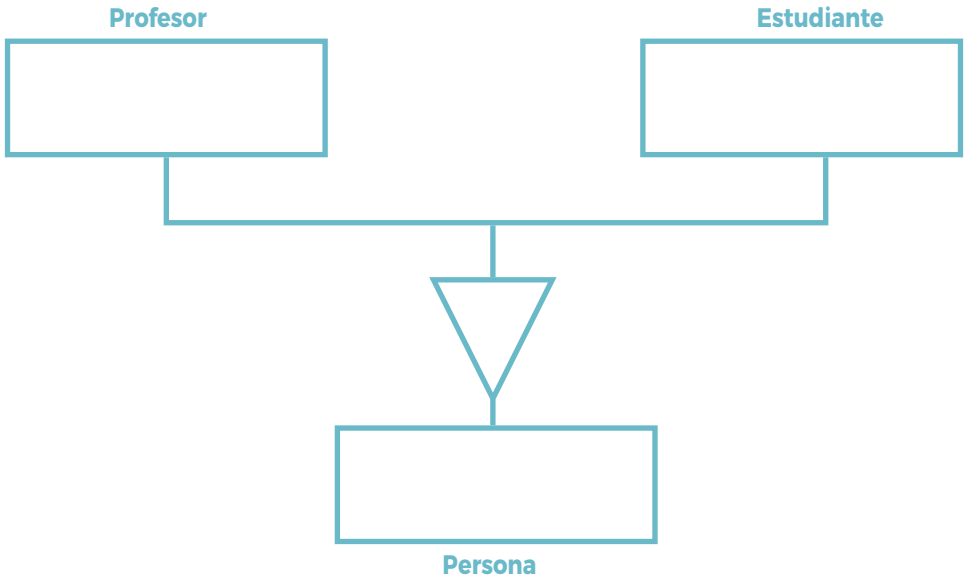
- **Generalización**: Se identifican patrones comunes en varias entidades y se agrupan en una entidad superior. → *de lo específico a lo general*.
- **Especialización**: Se parte de una entidad general y se definen entidades más específicas. → *de lo general a lo específico*.

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8.2.1. Generalización

Es el proceso mediante el cual **varias entidades específicas se agrupan en una entidad más general**, identificando atributos y relaciones comunes.

- **Ejemplo**:
  - » *Entidades: Profesor, Estudiante*
  - » *Generalización: Persona (con atributos comunes como nombre, DNI, fecha_nacimiento)*
- **Notación**: Flecha (triángulo) desde las entidades específicas hacia la entidad general.

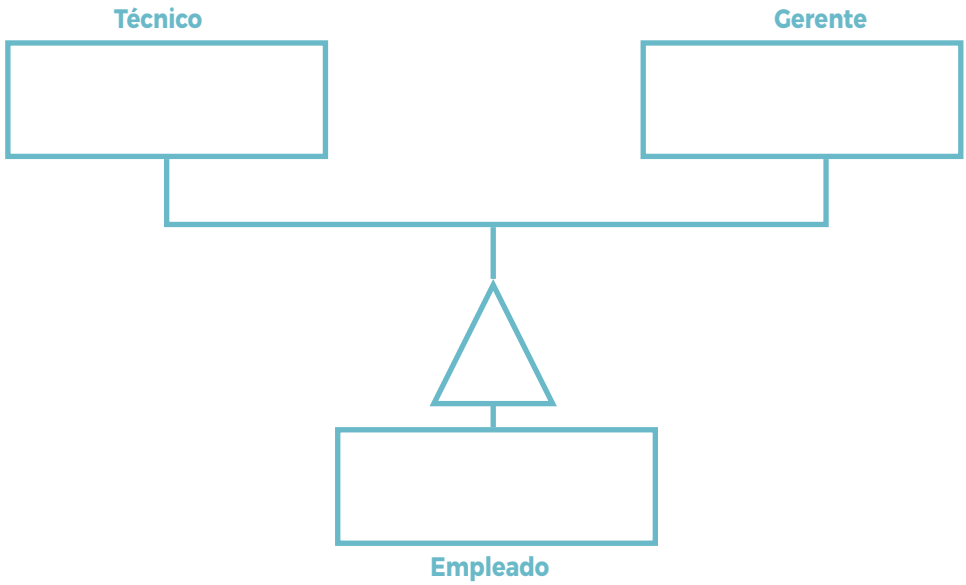


8.2.2. Especialización

Es el proceso inverso a la generalización, **consiste en definir entidades más específicas a partir de una entidad más general.**

- **Ejemplo:**
  - » *Entidad general: Empleado*
  - » *Especializaciones: Gerente, Técnico, Administrativo*
- **Criterios de especialización**: puede hacerse por valores (por ejemplo, tipo de contrato) o manualmente según el modelo de negocio.
- **Notación**: Flecha (triángulo) que va desde la entidad general hacia las especializadas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



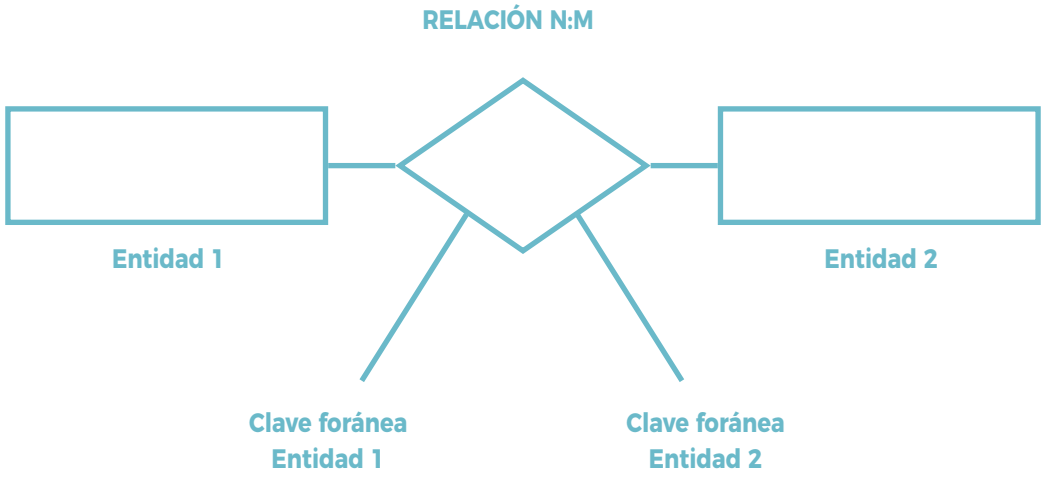
8.2.3. Entidades asociativas y de agregación en el Modelo E/R extendido

Tanto las **entidades asociativas** como las de **entidades de agregación** son mecanismos del **modelo entidad-relación extendido** que permiten representar estructuras más complejas y situaciones del mundo real que no pueden modelarse con relaciones simples.

Entidades asociativas: Una entidad asociativa es una entidad especial que se crea para representar una relación muchos a muchos (N:M) entre dos o más entidades. Esta entidad almacena información propia de la relación que no pertenece exclusivamente a ninguna de las entidades originales.

- Se utiliza porque en bases de datos relacionales **no se permite implementar directamente relaciones N:M**, por lo que se descompone en dos relaciones 1:N mediante una entidad intermedia.
- **Ejemplo**:
  - *Entidades: Estudiante y Asignatura.*
  - *Relación: un estudiante puede cursar muchas asignaturas y una asignatura puede tener muchos estudiantes.*
  - *Entidad asociativa: Matrícula con atributos como fecha, nota.*
- **Representación**: La entidad asociativa tiene claves foráneas a cada una de las entidades relacionadas y puede incluir atributos adicionales.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Agregación: La **agregación** es una técnica que permite **tratar una relación como si fuera una entidad** para **asociarla a otra entidad o relación**, y así **modelar jerarquías o niveles superiores de abstracción**.

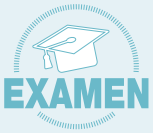
- Es útil cuando se necesita **añadir atributos a una relación** o **establecer relaciones entre una relación y otra entidad**.
- **Ejemplo**:
  - » Relación: Asignación entre Profesor y Proyecto.
  - » Si se quiere representar que un Departamento supervisa cada Asignación, entonces no se puede conectar directamente Departamento con Asignación, ya que esta es una relación.
  - » Solución: se **agrega** la relación Asignación y se conecta con Departamento.



- Usa entidades asociativas cuando una relación N:M necesita atributos adicionales y debe implementarse en un SGBD relacional.
- Usa agregación cuando quieres relacionar una relación con otra entidad, representando relaciones de nivel superior.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

9. NORMALIZACIÓN



La **normalización** es el proceso de organizar los datos en una base de datos para **eliminar redundancias** y mejorar la **integridad de los datos**. **Métrica v3** sigue las reglas de normalización estándar para garantizar que la estructura de la base de datos sea eficiente y escalable.

¿Qué es Métrica v3?

La **métrica v3** es una **metodología para estimar el tamaño funcional del software** en proyectos desarrollados para la Administración Pública en España. Es la evolución de versiones anteriores (como métrica v2), y forma parte de la documentación del **Centro Criptológico Nacional (CCN)**, aunque su uso es más extendido en el ámbito del desarrollo de sistemas informáticos en el Sector Público español.

Es una **metodología de planificación, análisis, diseño y desarrollo de sistemas de información**, que sirve para:

- Gestionar correctamente el ciclo de vida de un sistema de información.
- Estandarizar procesos en proyectos de software.
- Facilitar la contratación pública y la subcontratación técnica.
- Integrarse con otras metodologías (ágiles, PMI, etc.), especialmente en entornos formales.

Métrica V3 se aplica sobre el diseño de la base de datos de forma **estructurada y progresiva** en varias fases de la metodología, especialmente en estas dos:

Fase	Proceso	Relación con base de datos
Diseño del Sistema de Información (DSI)	Diseño lógico de datos	Se definen las entidades, relaciones, atributos y restricciones, independientemente del SGBD.
Diseño Técnico del Sistema (DTS)	Diseño físico de datos	Se traduce el modelo lógico al modelo físico específico del SGBD (tipos de datos, índices, particiones, normalización/denormalización, etc.).

MÉTRICA v3 define entregables concretos relacionados con el diseño de la base de datos:

Producto	Contenido
Diccionario de Datos	Definición formal de cada dato: nombre, descripción, tipo, longitud, valores permitidos, etc.
Modelo de Datos Lógico	Entidades, relaciones, cardinalidades, atributos y claves. Normalizado (3FN o superior).
Modelo de Datos Físico	Tablas, columnas, tipos de datos físicos, claves primarias y foráneas, índices, restricciones.
Especificación de Acceso a Datos	Consultas, inserciones, actualizaciones y borrados previstos. Incluye vistas, procedimientos, etc.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

9.1. NORMALIZACIÓN DE BASES DE DATOS. CONCEPTO Y OBJETIVOS

La normalización es un proceso clave en el diseño de bases de datos relacionales que busca minimizar la **redundancia** y mejorar la **integridad** de los datos. A través de un conjunto de reglas y principios, se estructuran las tablas de manera eficiente para evitar **anomalías en la inserción, actualización y eliminación** de datos.

La normalización es una técnica que organiza los datos dentro de una base de datos siguiendo una serie de **formas normales** para garantizar que la estructura sea óptima. Se basa en la identificación de **dependencias funcionales** entre los atributos de una tabla y en la eliminación de **redundancias** innecesarias.

Objetivos de la Normalización

1. **Eliminar la redundancia de datos**
2. **Prevenir anomalías en las operaciones**
3. **Mantener la integridad de los datos**
4. **Optimizar el almacenamiento y el rendimiento**



Importancia de la Normalización en Métrica v3

- Evita duplicación de datos y redundancia.
- Asegura coherencia e integridad referencial.
- Facilita la actualización y mantenimiento de los datos.

9.2. FORMAS NORMALES

El proceso de normalización se lleva a cabo a través de diferentes **formas normales (FN)**. Cada una impone reglas más estrictas para estructurar los datos de manera eficiente.

La **normalización** es un proceso teórico cuyo objetivo es **mejorar la estructura de las tablas de una base de datos relacional**, eliminando redundancias, asegurando la integridad de los datos y facilitando su mantenimiento.

Este proceso se basa en el cumplimiento de una serie de **formas normales**, que son niveles de calidad estructural. Cada forma normal impone unas reglas que, si se cumplen, permiten asegurar que la tabla está correctamente diseñada respecto a ciertos tipos de dependencias funcionales.

Las bases de datos diseñadas bajo **Métrica v3** deben cumplir con las siguientes **formas normales**:

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **1FN (Primera Forma Normal):** No debe haber grupos repetitivos ni atributos multivaluados.
- **2FN (Segunda Forma Normal):** Debe estar en 1FN y todos los atributos deben depender completamente de la clave primaria.
- **3FN (Tercera Forma Normal):** Debe estar en 2FN y eliminar las dependencias transitivas.
- **BCNF (Forma Normal de Boyce-Codd):** Versión más estricta de la 3FN, asegurando que cada determinante sea una clave candidata.

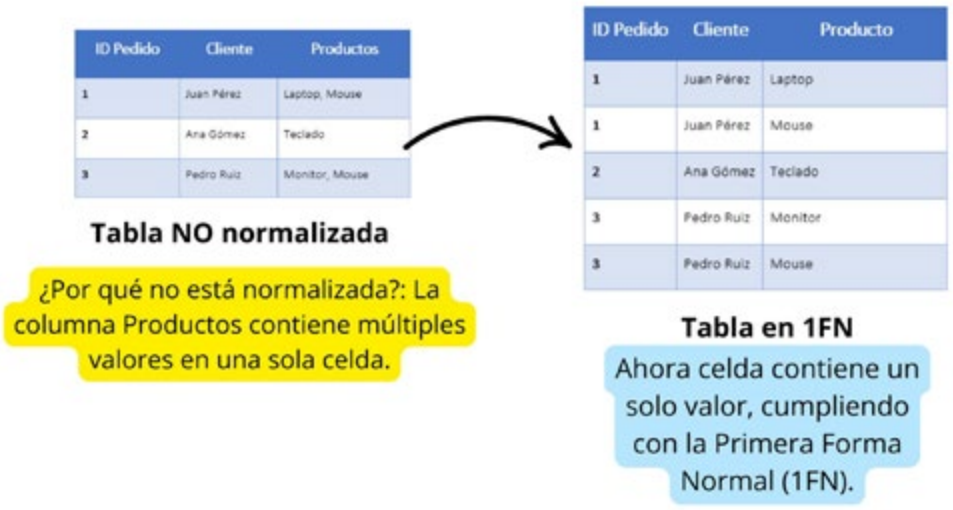
9.2.1. Primera Forma Normal (1FN)

La Primera Forma Normal constituye el nivel más básico de normalización y **establece que los atributos de una tabla deben contener valores atómicos, es decir, indivisibles**.

Esto implica que no se permiten columnas con listas de valores, estructuras repetitivas o anidadas. Además, **se requiere que todos los registros de una tabla tengan el mismo número de columnas y que cada campo pertenezca a un único dominio**.

Reglas:

- Los valores en cada columna deben ser **atómicos** (no divisibles).
- No deben existir **listas o conjuntos de valores** dentro de una misma celda.



9.2.2. Segunda Forma Normal (2FN)

La **Segunda Forma Normal** tiene como objetivo **eliminar** las **dependencias parciales**, es decir, aquellas en las que un **atributo no clave depende solo de una parte de la clave primaria compuesta**.

Para poder aplicar esta forma normal, **la tabla debe estar previamente en 1FN**. Al eliminar estas dependencias, se evita la repetición innecesaria de datos asociados a una parte de la clave, mejorando la integridad y reduciendo la redundancia en las tablas con claves compuestas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Reglas:

1. La tabla debe cumplir con **1FN**.
2. Todo **atributo no clave** debe depender **totalmente** de la clave primaria, no de una parte de ella.

Ejemplo de una tabla que está en 1FN pero no en 2FN.

Problema: La **Dirección** depende del **Cliente**, pero el identificador de la tabla es **ID_Pedido + Producto**.

ID Pedido	Producto	Cliente	Dirección
1	Laptop	Juan Pérez	Calle 123
1	Mouse	Juan Pérez	Calle 123
2	Teclado	Ana Gómez	Av. 456

Solución: Dividir en dos tablas:

Tabla Pedidos:

ID Pedido	Cliente	Dirección
1	Juan Pérez	Calle 123
2	Ana Gómez	Av. 456

Tabla Detalle_Pedido:

ID Pedido	Producto
1	Laptop
1	Mouse
2	Teclado

Ahora, la **Dirección** depende solo del **Cliente** y no de la combinación de claves, cumpliendo con **2FN**.

9.2.3. Tercera Forma Normal (3FN)

La **Tercera Forma Normal** aborda las **dependencias transitivas**, que **se producen cuando un atributo no clave depende de otro atributo que tampoco es clave**, y no directamente de la clave primaria.

Una **tabla debe estar en 2FN para poder alcanzar la 3FN.** Esta forma normal persigue una estructura lógica más pura, donde todos los atributos dependan únicamente de la clave primaria, evitando así inconsistencias y redundancia en la representación de los datos.

Reglas:

1. La tabla debe estar en **2FN**.
2. No debe haber **dependencias transitivas** (es decir, que un atributo dependa de otro que no sea clave primaria).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo de una tabla en 2FN pero no en 3FN.

Problema: Nombre_Curso depende de **ID_Curso**, y **ID_Curso** depende de **ID_Estudiante**.

ID_Estudiante	Nombre	ID_Curso	Nombre_Curso
1	Juan	101	Matemáticas
2	Ana	102	Historia

Solución: Separar en dos tablas:

Tabla Estudiantes:

ID_Estudiante	Nombre	ID_Curso
1	Juan	101
2	Ana	102

Tabla Cursos:

ID_Curso	Nombre_Curso
101	Matemáticas
102	Historia

Ahora, la **dependencia transitiva** se ha eliminado y se cumple la **3FN**.

9.2.4. Forma Normal de Boyce-Codd (BCNF)

La **Forma Normal de Boyce-Codd** es una **versión más rigurosa de la 3FN** que refuerza el principio de dependencia total de los atributos respecto a claves.

Establece que, **para toda dependencia funcional no trivial en una tabla, el determinante debe ser una superclave**. Aunque la mayoría de las tablas en 3FN también cumplen BCNF, esta forma se hace necesaria en casos especiales donde existen anomalías provocadas por claves alternativas o superposiciones funcionales. Aplicarla garantiza una mayor solidez estructural en el diseño lógico.

BCNF es una versión más estricta de **3FN**. Se usa cuando una tabla tiene **dependencias funcionales múltiples**.

Reglas:

1. La tabla debe cumplir con **3FN**.
2. Cada determinante (atributo que define otros atributos) debe ser una **clave candidata**.

Ejemplo:
Si una tabla tiene claves compuestas y una de ellas no es clave primaria, se puede dividir para cumplir BCNF.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

9.2.5. Cuarta Forma Normal (4FN)

La **Cuarta Forma Normal** se centra en eliminar las **dependencias multivaluadas** dentro de una relación.

Reglas:

1. Cumple la **BCNF**.
2. No contiene dependencias multivaluadas no triviales.

Una **dependencia multivaluada** se da cuando un atributo A determina de forma independiente a dos atributos B y C, que no están relacionados entre sí, generando combinaciones cruzadas innecesarias. Esto puede llevar a redundancias y anomalías similares a las que se intentan evitar con las dependencias funcionales.

Para resolverlo, se descompone la tabla original en varias relaciones independientes, donde cada una contiene una de las dependencias multivaluadas.

Ejemplo: una persona que tiene varios títulos y habla varios idiomas. Si se modela todo en una única tabla, se combinan títulos e idiomas de forma innecesaria. Lo correcto es separar en dos tablas: una con los títulos y otra con los idiomas, ambas referenciando a la persona.

9.2.6. Quinta Forma Normal (5FN) o Forma Normal de Proyección-Unión

La **Quinta Forma Normal** busca eliminar las **dependencias de unión** no justificadas.

Reglas:

1. Está en 4FN.
2. No existen **anomalías de unión**: es decir, si se ha descompuesto una tabla en varias y su unión natural no reproduce exactamente la tabla original, entonces la descomposición no es válida.

La 5FN se aplica en casos muy específicos donde una tabla puede descomponerse en varias sin perder información y sin introducir inconsistencias, pero solo si **las relaciones parciales entre los atributos no implican pérdida de datos** al reconstruir la tabla.

Ejemplo: una empresa que tiene contratos entre empleados, proyectos y localizaciones. Si los datos se modelan mal, puede que no se pueda reconstruir correctamente qué empleado está asignado a qué proyecto en qué localización, a partir de las relaciones parciales.

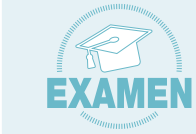
En la siguiente tabla describe el resumen de condiciones para cada una de las formas normales:

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Forma Normal	Condición 1	Condición 2	Propiedades
1FN (Primera Forma Normal)	No debe haber grupos repetitivos.	No debe haber atributos multivaluados.	- Cada columna tiene valores atómicos (indivisibles). - Todos los valores en una columna son del mismo tipo. - Cada fila es única e identificable con una clave primaria.
2FN (Segunda Forma Normal)	Debe estar en 1FN.	No debe haber dependencias parciales.	- Se eliminan dependencias parciales dividiendo la tabla. - Las columnas no clave dependen completamente de la clave primaria.
3FN (Tercera Forma Normal)	Debe estar en 2FN.	No debe haber dependencias transitivas.	- Se eliminan dependencias transitivas. - Las columnas no clave dependen únicamente de la clave primaria.
BCNF (Forma Normal de Boyce-Codd)	Debe estar en 3FN.	Cada determinante debe ser una clave candidata.	- Mejora la 3FN asegurando que no existan dependencias funcionales donde una columna no clave determine otra.
4FN (Cuarta Forma Normal)	Debe estar en BCNF.	No debe haber dependencias multivaluadas.	- Se eliminan dependencias multivaluadas dividiendo la tabla.
5FN (Quinta Forma Normal o Forma Normal de Proyección-Unión)	Debe estar en 4FN.	No debe haber dependencias de unión sin pérdida.	- Se eliminan dependencias de unión mediante descomposición sin pérdida de información.

9.3. DEPENDENCIA FUNCIONAL



En el contexto de bases de datos relacionales, se dice que existe una **dependencia funcional** entre dos conjuntos de atributos **X** e **Y** en una relación **R** (se denota como **X** →**Y**) si, para cada valor de **X**, hay **un único valor asociado de Y**.

Es decir, **X determina a Y**:**Si dos filas de una tabla tienen el mismo valor para X, entonces deben tener el mismo valor para Y.**

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo: Supongamos una tabla de empleados:

DNI	Nombre	Departamento
12345678	Laura	Compras
87654321	Marcos	Finanzas

Aquí se cumple la dependencia:

- **DNI → Nombre**
- **DNI → Departamento**

Porque **el DNI identifica de forma única** a cada empleado y, por tanto, su nombre y departamento.

Tipos de dependencias funcionales

Tipo	Descripción
Total	Todos los atributos de Y dependen completamente de todos los de X.
Parcial	Y depende solo de una parte de X (cuando X es una clave compuesta).
Transitiva	X → Z porque X → Y y Y → Z.
Trivial	Si Y está contenido dentro de X (ej. X = {A, B}, Y = {A}).
No trivial	Cuando Y no está incluido en X.

Aplicación en la normalización

La normalización busca eliminar **dependencias parciales y transitivas** para que las relaciones sean más sólidas y eviten anomalías de inserción, actualización o borrado.

- En **2FN**, se eliminan **dependencias parciales**.
- En **3FN**, se eliminan **dependencias transitivas**.
- En **BCNF**, se requiere que **todo determinante sea una superclave** (más estricto que 3FN).

9.4. DEPENDENCIA FUNCIONAL TRANSITIVA



Una **dependencia funcional transitiva** ocurre cuando **un atributo no clave** depende de **otro atributo no clave**, y este a su vez **depende de la clave primaria**.

Formalmente, se dice que en una relación existe una dependencia transitiva si se cumple:

Si **A** → **B** y **B** → **C**, entonces **A** → **C** es una **dependencia transitiva**, siempre que **C no** sea atributo clave ni esté incluido en A.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

¿Por qué es un problema?

Las dependencias transitivas **no permiten que se cumpla la Tercera Forma Normal (3FN)**. Esto puede provocar:

- **Redundancia de datos**
- **Inconsistencias** al actualizar, insertar o eliminar registros
- Mayor dificultad para mantener la integridad de la base de datos

Ejemplo:

Supón una tabla con los siguientes atributos:

EmpleadoID (PK)	DepartamentoID	NombreDepartamento

Dependencias:

- **EmpleadoID → DepartamentoID** (cada empleado pertenece a un departamento)
- **DepartamentoID → NombreDepartamento** (el nombre depende del departamento)

Entonces, por transitividad:

- **EmpleadoID → NombreDepartamento**

Esto es una **dependencia transitiva**, ya que el nombre del departamento **no depende directamente** del empleado, sino **a través del DepartamentoID**.

¿Cómo se soluciona?

Se **normaliza** la relación dividiéndola en dos tablas:

1. Una tabla con EmpleadoID, DepartamentoID
2. Otra tabla con DepartamentoID, NombreDepartamento

Así se eliminan las redundancias y se asegura una estructura más coherente.

9.5. DEPENDENCIA DE REUNIÓN (JOIN DEPENDENCY)

En la teoría de bases de datos, una **dependencia de reunión** (en inglés, *join dependency*) se produce cuando una **relación (tabla) puede ser reconstruida sin pérdida de información** a partir de una combinación (mediante unión natural) de **algunas de sus proyecciones** (subconjuntos de columnas).

Formalmente, una relación R cumple una dependencia de reunión si puede ser expresada como la **unión de varias proyecciones** (por ejemplo, R1, R2, R3) y dicha unión **reproduce exactamente la relación original**, sin crear tuplas espurias ni perder información.

Relación con la Quinta Forma Normal (5FN)

La dependencia de reunión es el fundamento teórico de la **Quinta Forma Normal (5FN)**, también conocida como **forma normal de proyección-unión**.

Una relación está en **5FN** si **todas las dependencias de reunión son consecuencias de claves**.

Esto significa que **no hay ninguna descomposición posible** en más de dos relaciones que permita reconstruir la tabla sin pérdida de información **salvo las que están justificadas por claves**.

Supera. By thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Resumen:

- La **dependencia de reunión** garantiza que una descomposición de una tabla **no pierde información**.
- Es esencial cuando trabajamos con relaciones que involucran **más de dos atributos clave** (relaciones ternarias o superiores).
- Se aplica sobre todo en modelos **altamente normalizados**, aunque en la práctica su uso es menos frecuente que las formas normales anteriores (1FN a 3FN y BCNF).

9.6. EJEMPLO ANÁLISIS FORMAS NORMALES

Dada una relación compuesta por los atributos **A, B, C y D**, se sabe que:

- La **única clave candidata** es el par **(A, B)**, es decir, la clave principal es compuesta.
- Existen las siguientes **dependencias funcionales**:
  - (A, B) → C
  - C → D

A partir de esta información, se evalúa en qué forma normal se encuentra la tabla.

1ª Forma Normal (1FN)

Una tabla cumple la 1FN si todos los atributos tienen **valores atómicos** (no multivaluados ni compuestos). Como no se especifica lo contrario, se asume que todos los campos contienen valores indivisibles.

La tabla cumple la Primera Forma Normal (1FN).

2ª Forma Normal (2FN)

Para cumplir la 2FN, se deben cumplir dos condiciones:

1. La tabla está en 1FN.
2. **No existen dependencias funcionales parciales** de atributos no clave respecto de **una parte** de la clave principal (esto aplica si la clave es compuesta).

En este caso:

- La clave es **(A, B)**.
- El atributo **C depende de (A, B)**, no solo de A o de B.
  - Dependencia: **(A, B) → C** → C **depende de toda la clave**, **no** hay dependencia parcial.
- No hay ninguna dependencia parcial.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

La tabla cumple también la Segunda Forma Normal (2FN).

3ª Forma Normal (3FN)

La 3FN exige:

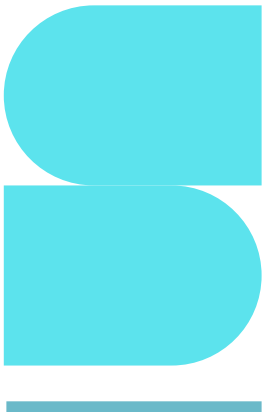
1. Estar en 2FN.
2. Que **no haya dependencias funcionales transitivas** de atributos no clave respecto de la clave principal.

Aquí:

- (A, B) → C
- C → D
- Por tanto, (A, B) → D **por transitividad**
- Como **D no es clave** y depende de C (que tampoco lo es), esta dependencia transitiva **viola la 3FN**.

La tabla no cumple la Tercera Forma Normal (3FN).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.